

Mock autotests — setup and run

This section describes how to bring up a local Omega container running with the deterministic LLM mock and run the `test_*_mock.py` suite against it.

For the live-provider counterpart (real LLM, grading scheme, parameters table) see [Autotests/README_live.pdf](#).

1. Prerequisites

- Docker engine on the host.
- Repository checked out, working from its root.
- Python virtual environment under `Autotests/venv` with `pytest` installed.

2. Build the local image

The mock infrastructure is part of the source tree, so the image must be built locally rather than pulled from the registry.

```
docker build -t omega:mock .
```

3. Start the container with the Test provider

Use the `scripts/omega` wrapper. It takes care of `--init`, `--user`, the `--tmpfs` mounts, `--security-opt no-new-privileges`, the persistent memory volume, and the `commchannel/provider/embeddingprovider` arguments, so the test setup stays in sync with how the agent is started in production and in CI.

The container connects back to the host on TCP port 9765 to reach the mock LLM controller and on TCP port 9766 to reach the test communication channel server. `TEST_SERVER_IP` must hold the host IP that is reachable from inside the container; under the default Docker bridge this is `172.17.0.1`.

```
env TEST_SERVER_IP=172.17.0.1 ./scripts/omega start -s 0000 -p Test -t test -d omega:mock
```

Notes:

- `-t test` selects the in-process test communication channel; messages travel over a TCP RPC between the container and the host fixture, not over IRC, Telegram, or Slack.
- `-p Test` selects the mock LLM dispatcher.
- `-s 0000` sets `OMEGA_AUTH_SECRET` inside the container.
- `-d omega:mock` points at the local image built in step 2.
- `TEST_SERVER_IP=172.17.0.1` is the host's docker-bridge address used by both the mock LLM provider and the test channel client.
- The container is created with the name `omega` (the script default).

Wait until the agent loop is up. The first runtime `CHARS_SENT:` line (with a byte count after the colon) in the container log marks the end of `initChannels` / `initMemory` and the start of real iterations; the bare `CHARS_SENT:` string also appears earlier as part of the MeTTa source dump, so match on the numeric form to avoid a premature exit:

```
until docker logs omega 2>&1 | grep -qE "CHARS_SENT: [0-9]+"; do sleep 2; done
```

4. Configure the test environment

Export the variables the test harness reads.

```
export OMEGA_CONTAINER=omega
export OMEGA_GIT_TOKEN=<github_pat> # only required by test_git_push_to_remote_mock
```

Variable	Required	Description
----------	----------	-------------

OMEGA_CONTAINER	Yes	Container name passed to <code>docker exec</code> from the harness. Must equal the container name created in step 3 (<code>omega</code> when using the script).
OMEGA_GIT_TOKEN	No	GitHub PAT used by <code>test_git_push_to_remote_mock</code> . The test is skipped if this variable is unset.

5. Run the suite

```
cd Autotests
source venv/bin/activate
pytest -s -v mock/test_*_mock.py
```

The `LlmMockController`, `CommMockServer`, and the OpenClaw Gateway stub are provided by session-scoped fixtures in `mock/conftest.py`, so all three are started once per pytest session. Expected output: 42 passed, minus the ones whose optional variables are unset - `test_git_push_to_remote_mock` skips without `OMEGA_GIT_TOKEN`, and the six OpenClaw tests skip without `OMEGA_OPENCLAW_TOKEN` (see “OpenClaw plugin” below).

5a. OpenClaw plugin (`test_openclaw_delegate_mock.py`)

`mock/test_openclaw_delegate_mock.py` exercises the `openclaw` plugin (`plugins/openclaw/`), whose `delegate-task-to-openclaw-agent` skill talks HTTP to an **OpenClaw Gateway**. The suite brings its own: the session-scoped `openclaw_gateway` fixture in `conftest.py` starts the stub in `mock/ocgw_stub.py`, which implements the same OpenResponses `POST /v1/responses` contract on port 18789. No external Gateway, no manual setup, and the file runs under a plain `pytest mock/test_*_mock.py` like everything else here.

The stub is what makes the interesting cases reachable at all. Its behaviour is steered by markers inside the delegated task, so a test can ask for a Gateway that stalls, refuses, or answers with nothing:

Marker in the task text	Gateway behaviour
OCGW_SLEEP:<seconds>	holds the reply, used to prove delegation does not block the agent loop
OCGW_503:<n>	answers 503 with <code>Retry-After</code> for the first <n> calls, then succeeds
OCGW_UNAUTHORIZED	answers 401 regardless of the token
OCGW_NOTEXT	answers 200 with a reasoning-only output and no visible text
Reply with exactly: X	answers exactly X

The stub also records every request it served, so a test can assert what the Gateway actually saw rather than inferring it from the agent’s history: whether Nginx injected the `Authorization` header, and whether two delegations really landed in separate sessions.

Two things must still be true of the container, because the plugin is configured at launch:

1. **Start it with `-g`**, which points Nginx’s `/openclaw/` proxy at the stub and sets `openClawEnabled/openClawURL` - no `config/config.yaml` edit needed:

```
env TEST_SERVER_IP=172.17.0.1 OMEGA_OPENCLAW_TOKEN=<token> \
./scripts/omega start -s 0000 -p Test -t test -d omega:mock -g "http://172.17.0.1:18789"
```

Use the address the container reaches the host on: `172.17.0.1` under the default Docker bridge, `host.docker.internal` in CI. Starting the container by hand instead of through the script? Pass `openclaw_url=<gateway URL>` as a container argument, same as `openaiapi_url=` (see `entrypoint.sh`).

2. **Export `OMEGA_OPENCLAW_TOKEN`**. The stub accepts exactly this value, so the tests fail if Nginx does not inject it. Nginx reads it before the agent’s environment is scrubbed, which is also what gives `test_credentials_scrubbed_mock.py` a real token to catch a leak of - with the variable unset, that assertion passes without proving anything.

If the token is unset the suite skips, since the plugin cannot be exercised at all. If the token is set but the container

was started without `-g`, the fixture fails instead of skipping: a token with a disabled plugin means the run would look green while testing nothing.

6. Tear down

```
./scripts/omega clean
```

This removes the `omega` container and the `omega-memory` volume created by the script in step 3.

Tests description

All 42 tests follow the same pattern: the test registers a fixed mock-LLM answer for the prompt via `llm.set_answer(prompt, response)`, delivers the prompt to the agent over the test channel via `comm.send_message(prompt)`, then verifies the resulting skill calls and side effects (filesystem, `history.metta`, ChromaDB, docker logs). The OpenClaw group additionally drives the stub Gateway, whose replies are real rather than scripted. Because the LLM is deterministic, no `try_with_clarification` retries are needed; every test either passes on the first attempt or fails outright.

Creating files

1. test_create_file_mock.py

Creates `/tmp/testcat/hello.txt` containing exactly `Hello`.

- Mock answer: `(shell "mkdir -p /tmp/testcat") (write-file "/tmp/testcat/hello.txt" "Hello")`.
- Checks: directory exists, file exists, permissions start with `-rw`, mtime $\geq$ test start, content is `Hello` (with or without trailing newline).

2. test_create_empty_file_mock.py

Creates an empty `/tmp/test_empty/hello.txt`.

- Mock answer: `(shell "mkdir -p /tmp/test_empty") (write-file "/tmp/test_empty/hello.txt" "")`.
- Checks: file is created, `cat` returns an empty string.

3. test_create_script_mock.py

Creates `/tmp/test_script/date.sh`, a shell script that prints the current date.

- Mock answer: `(shell "mkdir -p /tmp/test_script") (write-file "/tmp/test_script/date.sh" "#!/bin/bash\ndate\n") (shell "chmod +x /tmp/test_script/date.sh")`.
- Checks: file exists, is executable (`x` in permissions), harness runs it via `sh date.sh` and verifies the output contains the current year (UTC or local).

Editing files

4. test_edit_add_timestamp_mock.py

Appends the current timestamp as a new line to a pre-created `note.txt`.

- Mock answer: `(shell "date -Iseconds >> {TARGET_FILE}")`.
- Checks: mtime changed, last line of the file contains ≥ 4 digits.

5. test_edit_delete_line_mock.py

Deletes the second line from a pre-created 3-line file.

- Mock answer: `(shell "sed -i 2d {TARGET_FILE}")`.
- Checks: mtime changed, exactly 2 lines remain, lines 1 and 3 intact.

6. test_edit_append_line_mock.py

Appends a 4th line `Delta` to a pre-created 3-line file (Alpha / Bravo / Charlie).

- Mock answer: `(shell "printf '%s\\n' Delta >> {TARGET_FILE}").`
- Checks: mtime changed, 4 lines total, first 3 unchanged, 4th line equals Delta.

7. test_convert_format_mock.py

Converts `document.md` → `document.txt`, preserving textual content.

- Mock answer: `(shell "cp {SOURCE_FILE} {DEST_FILE}").`
- Checks: `.txt` exists, contains the keywords `My Title`, `Some paragraph text`, `item one`, `item two`.

Running shell scripts

8. test_run_error_script_mock.py

Runs a syntactically broken pre-created script and captures stdout and stderr to a file.

- Mock answer: `(shell "sh {SCRIPT_FILE} > {OUTPUT_FILE} 2>&1").`
- Checks: `output.txt` exists, contains the literal `start` (stdout) AND at least one of `error` / `syntax` / `unexpected` / `missing` / `not found` (stderr); container is still alive.

9. test_run_repeated_mock.py

Runs `dateupdate.sh` exactly 10 times in a row.

- Mock answer: ten consecutive `(shell "{SCRIPT_FILE}")` calls (one per run).
- Checks: `update.txt` exists with mtime ≥ start, has ≥ 10 lines, every line contains date-like digits.

Internet search

10. test_search_basic_mock.py

Sends prompt “What is SingularityNet? Search the web.”

- Mock answer: `(send "SingularityNet (SNET) is a decentralized AI marketplace founded by Ben Goertzel. Its native token AGIX powers a network of AI agents and services, and it is part of the broader ASI Alliance ecosystem.")`.
- Checks: the reply contains at least one of `singularitynet`, `agi`, `blockchain`, `decentralized`, `marketplace`, `goertzel`.

11. test_search_weather_mock.py

Returns a synthetic Valencia weather reply (the live variant cross-checks against open-meteo; the mock variant fixes a synthetic reference `REF_TEMP_C = 18.0` and stays fully offline).

- Mock answer: `(send "Current weather in Valencia, Spain: about 18.0°C.")`.
- Checks: the reply contains a plausible Celsius number (range [-20; 50]); at least one of those numbers is within ±10 °C of the synthetic reference 18.0 °C.

12. test_search_invalid_mock.py

Asks about a gibberish string.

- Mock answer: `(send "No results found for <gibberish>. The string appears to be gibberish, no meaningful matches.")`.
- Checks: the reply contains a negation phrase (`no results`, `not found`, `gibberish`, `nonsense`, `no meaning`, `unknown`, ...).

Memory

13. test_memory_chromadb_mock.py

Requests the agent to remember a fact tagged with marker `CI-SMOKE-<run_id>`.

- Mock answer: `(remember "Unique smoke marker CI-SMOKE-<run_id> was emitted by CI.")`.

- Checks: `(remember ...)` was invoked with the marker; vector count in the `embeddings` table of `chroma.sqlite3` grew by ≥ 1 .

14. test_memory_history_mock.py

Sends “Acknowledge with one short line that you received marker `<run_id>.`” and verifies the entry in `history.metta`.

- Mock answer: `(send "Acknowledged marker <run_id>.")`.
- Checks: an s-exp record referencing `REQ-<run_id>` appears in history; the agent issued `(send ...)`; file mtime and size grew.

Skills

15. test_skill_metta_mock.py

Asks the agent to evaluate a short MeTTa expression and report the result.

- Mock answer: `(metta "(+ 2 2)") (send "The metta skill evaluated (+ 2 2) and returned 4.")`.
- Checks: `(metta ...)` was invoked; the agent then issued a `(send ...)`. Semantic correctness of the MeTTa expression is not checked; the goal is to exercise the skill.

16. test_skill_pin_mock.py

Gives a multi-step task (“restarting servers $\alpha \rightarrow \beta \rightarrow \gamma$, just finished α ”) and expects the agent to track progress with `pin`.

- Mock answer: `(pin "Server restart progress: alpha done; beta and gamma pending.") (send "Tracking: alpha done, beta and gamma pending.")`.
- Checks: `(pin ...)` was invoked whose argument references either the `run_id` or one of the keywords `step / alpha / beta / gamma / restart / server / done`; the agent acknowledged via `(send ...)`.

Working with git

17. test_git_pull_public_mock.py

Agent clones a public repository over anonymous HTTPS, no token.

- Mock answer: `(shell "rm -rf {TARGET_DIR} && git clone {remote} {TARGET_DIR}")`.
- Checks: `.git/` appears, HEAD points to a real commit, ≥ 1 tracked file in HEAD, origin matches the expected remote URL (normalized, trailing `/` and `.git` ignored).

18. test_git_local_commit_mock.py

Agent runs `git init`, `git add`, `git commit` locally inside the container.

- Mock answer: chain of `(shell "git -C {TARGET_DIR} init") (shell "...write file...") (shell "git -C {TARGET_DIR} add -A") (shell "git -C {TARGET_DIR} commit -m 'add hello <run_id>')`.
- Checks: HEAD has at least one commit, commit subject contains the `run_id` (warning, not failure), the file is present in the tree.

19. test_git_push_to_remote_mock.py

Agent clones a remote, creates branch `qa/run-<id>`, adds a file, commits, and pushes.

- Mock answer: single `(shell "rm -rf ... && git clone ... && cd ... && git checkout -b ... && printf ... > <file> && git add -A && git commit -m '...' && git push -u origin ...")`.
- Parameters via env vars: `OMEGA_GIT_TOKEN` (token; never appears in code) and `OMEGA_GIT_REMOTE` (default `https://github.com/OmegaSing/Test-Repopo`). Test is skipped if the token variable is unset.
- Checks: branch present on remote (GitHub API 200), file present on branch, the shell call included `git push`, credentials wiped on teardown.

Multi-skill tests

20. test_run_create_dirs_mock.py

Agent writes `makedirs.sh` and runs it. The script must create `test1`, `test2`, `test3` inside `/tmp/test_dirs/`.

- Mock answer: `(write-file "{SCRIPT_PATH}" "#!/bin/bash\nmkdir -p ../test1 ../test2 ../test3\n") (shell "chmod +x {SCRIPT_PATH}") (shell "{SCRIPT_PATH}")`.
- Checks: all three directories exist with fresh mtimes; agent invoked `(write-file ...)` referencing `makedirs.sh`; agent invoked `(shell ...)` to run the script. Diagnostics print `wf=<count>`, `sh=<count>`, `perms=<...>` to make stalls obvious.

21. test_memory_episode_mock.py

Two-turn flow: tells the agent that the user's dog Barney lost a baby tooth, waits 5 seconds, then asks to recall when this happened.

- Turn 1 mock answer: `(remember "Barney the dog lost his first baby tooth at the vet today.")`.
- Turn 2 mock answer: `(query "Barney tooth") (send "Barney the dog lost his first baby tooth on <YYYY-MM-DD>. The milestone is recorded in my notes.")`.
- Checks (turn 1): `(remember ...)` was invoked whose argument contains `tooth` or `Barney`. Checks (turn 2): `(query ...)` or `(episodes ...)` was invoked; the reply contains at least one of `dog` / `tooth` / `lost`; the reply contains the captured seed date in `YYYY-MM-DD` format.

22. test_skill_query_mock.py

Two-turn flow: plant a unique color (`azure-<run_id>`) via `remember`, wait for embeddings to settle, then ask the agent to recall it via `query` (embedding lookup, not timestamp lookup).

- Turn 1 mock answer: `(remember "My favorite color is azure-<run_id>.") (send "Stored: favorite colour is azure-<run_id>.")`.
- Turn 2 mock answer: `(query "favorite color") (send "Your favorite color is azure-<run_id>.")`.
- Checks (turn 1): `(remember ...)` carried the secret color. Checks (turn 2): `(query ...)` was invoked; the reply mentions the secret color verbatim.

23. test_skill_episodes_mock.py

Two-turn flow: send a message tagged with a unique keyword (no `remember`), capture the timestamp, then ask the agent to use `episodes` (timestamp lookup, not `query`) to recall what was discussed at that earlier time.

- Turn 1 mock answer: `(send "Acknowledged keyword <marker>.")`.
- Turn 2 mock answer: `(episodes "<seed_ts>") (send "The unique keyword was <marker>.")`.
- Checks (turn 1): the turn is recorded in `history.metta` with a timestamp. Checks (turn 2): `(episodes ...)` was invoked for the seed timestamp; the reply mentions the original marker.

24. test_complex_weather_flow_mock.py

Four-step pipeline: search NY weather → write `w.txt` with the forecast → write `p.sh` extracting the first Celsius number into `t.txt` → run `p.sh`. Because the mock controls only the LLM dispatch (the network-bound `search` skill is not exercised), the mocked response provides the forecast text directly.

- Mock answer: `(write-file "/tmp/wflow/w.txt" "New York tomorrow: clear, high 22 degrees Celsius.") (write-file "/tmp/wflow/p.sh" "#!/bin/bash\ngrep -oE '[0-9]+' /tmp/wflow/w.txt | head -1 > /tmp/wflow/t.txt\n") (shell "chmod +x /tmp/wflow/p.sh") (shell "/tmp/wflow/p.sh")`.
- Checks: `w.txt` exists; history contains `(write-file ...)` referencing `w.txt`; `p.sh` exists with executable bit; history contains `(write-file ...)` or `(shell ...)` referencing `p.sh`; `t.txt` exists; history contains `(shell ...)` running `p.sh`; `t.txt` content is a number in the range `[-60; 120]`; content length ≤ 40 characters.

Memory tiers and transitions

25. test_last_skill_results_visible_next_turn_mock.py

Verifies the one-iteration carry of `LAST_SKILL_USE_RESULTS`. Output of a skill call in turn N is exposed to the LLM at turn N+1 via this prompt section. The test does not require the agent to “behave intelligently”; it confirms the carry exists.

- Mock answer (turn 1): `(metta "(+ 1 1)")`.
- Checks: the docker log line `CHARS_SENT`: for the next iteration contains a `LAST_SKILL_USE_RESULTS` section that reflects the metta output.

26. test_memory_history_byte_window_truncation_mock.py

Verifies that `history.metta` is a sliding byte-window. A marker placed early in the trace, then pushed past the `maxHistory` boundary by a large follow-up entry, must remain in the file on disk yet be absent from the trailing `maxHistory` bytes (the slice fed back to the agent as `HISTORY`).

- Mock answer: an initial `(remember ...)` with the marker, followed by a sequence that emits enough bytes to evict it from the trailing window.
- Checks: marker present in `history.metta` on disk; marker absent from the trailing `maxHistory` bytes returned by `getHistory`.

27. test_memory_pin_window_visibility_mock.py

A `(pin ...)` emitted in turn 1 must land in `history.metta` and remain inside the agent's rolling `HISTORY` window when turn 2 fires.

- Turn 1 mock answer: `(pin "<marker>")`.
- Turn 2 mock answer: `(send "ack")`.
- Checks: the pin block is on disk; the pin block sits within the trailing `maxHistory` byte window at turn 2.

28. test_pin_invisible_within_iteration_mock.py

Negative test: a `(pin ...)` emitted within an iteration is NOT visible inside that same iteration's `HISTORY` context. The prompt is assembled before skill evaluation, so the pin block, written by `addToHistory` at the end of the iteration, only enters `HISTORY` at the next prompt-build.

- Mock answer: `(pin "<marker>")` followed by a `(send ...)`.
- Checks: the `CHARS_SENT` line carrying the `PROMPT` for the iteration that contained the pin does NOT contain the pin's unique marker; the next iteration's `CHARS_SENT` line does.

29. test_transition_episodes_after_eviction_mock.py

A marker pushed out of the trailing `maxHistory` window is still recoverable via the `episodes` skill (timestamp-based history scan).

- Turn 1 mock answer: a beacon `(send "<marker>")`; the test captures the timestamp.
- Turn 2 mock answer: ~35K bytes of padding designed to evict the beacon from the trailing `HISTORY` window.
- Turn 3 mock answer: `(episodes "<seed_ts>")` against the captured timestamp.
- Checks: `(episodes ...)` was invoked with the captured timestamp. The skill's return value itself is mock-irrelevant; the test exercises the path against history the agent can no longer see in `HISTORY`.

30. test_transition_metta_to_remember_mock.py

Tier-3 to Tier-2 transition. A reasoning conclusion produced inside an `AtomSpace` via `(metta ...)` is ephemeral by design; if the agent wants to keep it, it must call `(remember ...)` on the conclusion.

- Mock answer: a `(metta ...)` inference call AND a `(remember "<conclusion>")` call.
- Checks: both skill calls fired; ChromaDB vector count grew by exactly one.

31. test_transition_pin_to_remember_mock.py

Explicit working-memory to long-term-memory transition.

- Turn 1 mock answer: `(pin "<checklist>")` into working memory.
- Turn 2 mock answer: `(remember "<checklist>")`, committing the pin to long-term memory.
- Checks: both skill calls landed; ChromaDB vector count grew by exactly one (the working-memory item was promoted to the persistent embedding store).

Security

32. test_credentials_scrubbed_mock.py

Verifies that provider keys, channel tokens, and the auth secret are scrubbed from the agent process environment after startup. The agent runs as `nobody` under an entrypoint that rebuilds the environment from a fixed allowlist, so secrets passed into the container reach the proxy/entrypoint but never the agent process itself.

- Mock answer: `(shell "env > /tmp/omega38_agentenv.txt")` — the agent dumps its own environment (only the process owner can read it).
- Checks: `PATH` is present (positive control that the environment was captured); none of the forbidden secret variable names are present in the agent environment (`OMEGA_AUTH_SECRET`, the provider API keys `ANTHROPIC_API_KEY` / `ASI_API_KEY` / `ASIONE_API_KEY` / `OPENAI_API_KEY` / `OPENROUTER_API_KEY` / `OPENAIAPI_API_KEY`, and the channel tokens `TG_BOT_TOKEN` / `SL_BOT_TOKEN` / `MM_BOT_TOKEN`). A leak fails with `leaked into agent env: [...]`. The check is by variable name, so it is independent of the secret values.

OpenClaw plugin (test_openclaw_delegate_mock.py)

The Gateway is the stub started by the `openclaw_gateway` fixture - see “5a. OpenClaw plugin” above. Unlike every other test in this file, the mocked LLM only decides *to call* `delegate-task-to-openclaw-agent`; the skill itself is not scripted and makes a real HTTP call, so its result is genuine, not canned.

Delegation is asynchronous, so these tests assert two stages: the acceptance envelope returned immediately (captured with `write-file`), and the `OPENCLAW_RESULT ...` line the plugin appends to `history.metta` once the Gateway answers (polled with `wait_for_history_keyword`).

33. test_delegate_isolated_and_success_mock

Two phases:

- Phase 1 - mock answer: `(send "Delegating task <run_id>") (delegate-task-to-openclaw-agent "Reply with exactly: unused")`. Checks: the `send` argument does not swallow the following skill call (parser regression guard).
- Phase 2 - mock answer: `(metta (write-file "<out>.json" (delegate-task-to-openclaw-agent "Reply with exactly: PONG-<run_id>"))) (send "Delegation saved <run_id>")`. Checks: the acknowledging `send` lands within 30s; the saved envelope has `status: "accepted"`, a non-empty `id`, and a `task echo` containing `PONG-<run_id>`; an `id=<task id> status=ok` record later reaches history; that line sits inside a well-formed `("YYYY-MM-DD HH:MM:SS"` block, so the `episodes` skill can still parse the file.

The wait keys on the task id rather than on the `PONG-<run_id>` marker alone: history also stores the agent’s own response text, which quotes the `delegate-task-to-openclaw-agent "...PONG-<run_id>"` command verbatim, so the marker is present immediately and would satisfy a marker-only wait before any reply exists.

34. test_delegate_empty_message_mock

Delegates an empty message - no network call and no worker thread, since the skill rejects it before queueing:

- Mock answer: `(metta (write-file "<out>.json" (delegate-task-to-openclaw-agent ""))) (send "Empty delegation checked <run_id>")`.
- Checks: the agent doesn’t crash; the JSON result has `status: "error"`, `type: "invalid_input"`. Validation stays synchronous precisely so this failure is still reported in the same turn.

35. test_delegate_new_session_per_call_mock

Verifies the “new session per delegation” contract from the plugin README: two independent delegations in the same turn must not share a Gateway session:

- Mock answer: two `(metta (write-file ... (delegate-task-to-openclaw-agent "Reply with exactly: FIRST-<run_id>" / "SECOND-<run_id>")))` calls, then `(send "Both delegations saved <run_id>")`.
- Checks: both envelopes are accepted with different `id` values; an `id=<task id> status=ok` record for each of them later reaches history (keyed on the task id for the reason given under test 35); the run’s slice of history carries two distinct `responseId=` values. The scoping to this run’s window keeps ids left by test 35 from satisfying the check on their own.

36. test_delegate_stays_async_under_a_slow_gateway_mock

The one test that can tell an asynchronous delegation from a synchronous one. The rest of this group answers

against a Gateway that replies instantly, so a blocking implementation would clear their budgets too.

- Mock answer: (delegate-task-to-openclaw-agent "OCGW_SLEEP:30 Reply with exactly: SLOW-<run_id>") (send "Long delegation started <run_id>"), then a second, unrelated prompt answered with (send "STILL-ALIVE-<run_id>").
- Checks: the acknowledgement lands within 15s while the Gateway is still holding the reply for 30s, so a delegation that blocked the loop cannot pass; the unrelated prompt is answered before the Gateway releases; the slow reply still reaches history afterwards.

37. test_delegate_reports_gateway_rejection_mock

A Gateway rejection must reach the agent instead of disappearing, since the skill returns before the call is made.

- Mock answer: (metta (write-file "<out>.json" (delegate-task-to-openclaw-agent "OCGW_UNAUTHORIZED <run_id>"))) (send "Refused delegation checked <run_id>").
- Checks: the envelope still says accepted; an id=<task id> status=error record carrying 401 later reaches history; the agent stays responsive throughout.

38. test_delegate_retries_a_starting_gateway_mock

A Gateway that is still booting answers 503, which the worker retries up to STARTUP_RETRY_ATTEMPTS times.

- Mock answer: (metta (write-file "<out>.json" (delegate-task-to-openclaw-agent "OCGW_503:2 Reply with exactly: RETRIED-<run_id>"))) (send "Retry delegation started <run_id>").
- Checks: the agent acknowledges without waiting out the retries, so they run off the loop; the delegation eventually succeeds with status=ok; the stub recorded three attempts, two refused and one served, rather than a single call.

39. test_delegate_reports_a_reply_without_text_mock

A terminal response whose output holds only reasoning items carries no visible reply, and must not be reported as an empty success.

- Mock answer: (metta (write-file "<out>.json" (delegate-task-to-openclaw-agent "OCGW_NOTEXT <run_id>"))) (send "Empty reply checked <run_id>").
- Checks: the agent survives; an id=<task id> status=error record reaches history.

40. test_delegation_is_authenticated_by_the_proxy_mock

The delegation path is authenticated by Nginx rather than by the agent, checked at the receiving end.

- Mock answer: (delegate-task-to-openclaw-agent "Reply with exactly: TOKEN-<run_id>") (send "Token delegation sent <run_id>").
- Checks: the stub recorded at least one request whose Authorization header matched the token, and no request arrived without that header. Since the agent process never holds the token, a matching header can only have come from the proxy.

The other half of this property - that OMEGA_OPENCLAW_TOKEN never reaches the agent process - belongs to test_credentials_scrubbed_mock.py, which reads the environment the agent dumps for itself. Do not check it with dexec: docker exec starts a new process from the container's own configuration, which does carry the token by design, so such a check fails while the scrubbing works correctly.